

Práctica 1 - Lógica combinacional

Sistemas Digitales

Segundo Cuatrimestre 2026

1.

Para cada una de las siguientes igualdades:

1. Determinar si es verdadera para todos los valores de sus variables;
2. Justificar la respuesta mediante leyes del álgebra de Boole, nombrando cada ley usada;
3. Verificar la conclusión mediante una tabla de verdad completa.
4. Programar en SystemVerilog + HDL cada uno de los circuitos

1. $x = x \cdot y + x \cdot \neg y$
2. $(x + y) \cdot (x + \neg y) = x$
3. $x \cdot z = (x + y) \cdot (x + \neg y) \cdot (\neg x + z)$
4. $\neg(x + y \cdot z) = \neg x \cdot (\neg y + \neg z)$
5. $x \oplus (y \cdot z) = (x \oplus y) \cdot (x \oplus z)$

Para toda igualdad falsa, indicar al menos una combinación de entradas que funcione como contraejemplo.

2.

Una función booleana de dos entradas a y b queda completamente determinada al elegir su salida para cada una de las cuatro combinaciones posibles.

1. ¿Cuántas funciones booleanas distintas de dos entradas existen? Justificar sin enumerarlas primero.
2. Completar una tabla con las cuatro combinaciones de ab , en orden $00, 01, 10, 11$, para cada función siguiente:
 - f_1 vale 1 únicamente cuando ambas entradas son distintas;
 - f_2 vale 1 únicamente cuando $a = 0$;
 - f_3 vale 0 únicamente para la entrada 11 .
3. Simplificar las expresiones obtenidas cuando sea posible.
4. Explicar por qué el procedimiento del punto 3 permite expresar cualquier función booleana usando únicamente AND, OR y NOT.

3.

Para cada función dada a continuación:

1. Programar en SystemVerilog + HDL cada uno de los circuitos.
 2. Construir su tabla de verdad.
 3. Simplificarla mediante leyes booleanas.
1. $f(a, b, c) = a \cdot b + a \cdot \neg b + \neg a \cdot b \cdot c$
 2. $g(a, b, c) = \neg(a + b \cdot c)$
 3. $h(a, b, c) = (a + b) \cdot (a + \neg b) \cdot (\neg a + c)$

Para g , dibujar además una segunda implementación obtenida al aplicar De Morgan hasta que las negaciones afecten solamente variables.

4.

Las funciones $F(A, B, C)$ y $G(D, E, F)$ están definidas por las siguientes tablas.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

D	E	F	G
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Para cada función:

1. Escribir la Suma de Productos y Producto de Sumas.
2. Programar en SystemVerilog + HDL la Suma de Productos o Suma de Productos.
3. Simplificar ambas formas mediante leyes del álgebra de Boole.

Luego implementar una de las dos funciones en SystemVerilog. El módulo debe llamarse **funcion3** y tener tres entradas de un bit **a**, **b**, **c** y una salida de un bit **out**. Probarlo con un testbench que recorra las ocho combinaciones y que detenga la simulación con un error si alguna salida no coincide.

5.

Diseñar un bloque combinacional con una entrada de datos **data**, una selección **sel[1:0]** y cuatro salidas **out[3:0]**. La salida seleccionada debe tomar el valor de **data**; las otras tres deben valer 0:

sel[1]	sel[0]	Comportamiento
0	0	out[0] = data ; las demás, 0
0	1	out[1] = data ; las demás, 0
1	0	out[2] = data ; las demás, 0
1	1	out[3] = data ; las demás, 0

1. Expandir la especificación en una tabla de verdad de ocho filas que incluya `data`, `sel[1:0]` y las cuatro salidas.
2. Obtener una ecuación Suma de Productos para cada salida.
3. Implementar el módulo `demux1a4` en SystemVerilog, sin usar operadores aritméticos.
4. Escribir un testbench exhaustivo para las ocho combinaciones de entrada.

6.

Un codificador recibe `in[3:0]`. Una entrada es válida cuando exactamente uno de sus bits vale

1. Para una entrada válida, `code[1:0]` indica la posición del bit activo: `0001` produce `00`, `0010` produce `01`, `0100` produce `10` y `1000` produce `11`. La salida `valid` vale 1 solo en esos cuatro casos.

1. Completar las 16 filas de la tabla de verdad de `in`, `valid` y `code`. Cuando `valid = 0`, usar `--` para indicar que `code` es irrelevante para la especificación.
2. Obtener ecuaciones para `valid`, `code[1]` y `code[0]`.
3. Implementar `encoder4` en SystemVerilog. En esta primera versión, fijar `code = 2'b00` siempre que `valid = 0`, de modo que toda salida del hardware quede definida.
4. Verificar las 16 entradas. El testbench debe comparar `code` únicamente cuando `valid = 1`.
5. Modificar solo una copia destinada a simulación para producir `2'bxx` cuando `valid = 0`. Explicar qué error de diseño podría revelar ese valor y por qué no representa un tercer valor booleano que el circuito pueda almacenar.

7.

Un decodificador de 2 a 4 recibe `in[1:0]` y activa exactamente una salida:

<code>in[1]</code>	<code>in[0]</code>	<code>out[3]</code>	<code>out[2]</code>	<code>out[1]</code>	<code>out[0]</code>
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

1. Obtener una ecuación para cada salida y dibujar el circuito.
2. Implementar un módulo `decoder2a4` con esa interfaz.
3. Construir un segundo módulo `demux1a4_jerarquico` que instancie `decoder2a4`. Cada salida debe ser el AND entre `data` y la línea correspondiente del decodificador.
4. Construir también `demux1a4_plano`, sin instanciar otros módulos.
5. Usar un único testbench para comprobar que ambas versiones coinciden para las ocho entradas posibles.
6. Comparar ambas descripciones en términos de legibilidad, posibilidad de reutilización e interfaz. ¿La jerarquía necesariamente reduce la cantidad de compuertas obtenida luego de sintetizar?

8.

Diseñar un circuito con una entrada `in[N-1:0]`, un control `invert` y una salida `out[N-1:0]`. Si `invert = 0`, la salida debe ser igual a la entrada; si `invert = 1`, cada bit de la salida debe ser el complemento del bit correspondiente de la entrada.

1. Resolver primero la función de un único bit mediante una tabla de cuatro filas que considere el bit de entrada `in[i]` y `invert`. La misma función se replica luego para cada uno de los tres bits del caso $N = 3$.

- Obtener una ecuación para `out[i]` y reconocer qué compuerta la implementa.
- Implementar `inversor_controlado` en SystemVerilog con un parámetro entero `N` cuyo valor predeterminado sea 3.
- Verificar estas entradas para $N = 3$: $(invert, in) = (0, 011), (1, 011), (1, 100), (1, 101)$.

9.

Este ejercicio distingue los bits lógicos del modelo de simulación. Considere una señal `bus` conectada a dos posibles fuentes, `driver_a` y `driver_b`. Cada fuente puede entregar 0, 1 o desconectarse, representado por Z.

driver_a	driver_b	bus
Z	Z	Z
0	Z	0
1	Z	1
Z	0	0
Z	1	1
0	0	0
1	1	1
0	1	X
1	0	X

- Explicar por qué los dos últimos casos se muestran como X.
- Explicar por qué Z no significa 0 y qué representa físicamente una salida desconectada o en alta impedancia.
- Si `bus = X`, determinar el resultado que mostraría un simulador para `not bus`, `bus & 1'b0`, `bus & 1'b1`, `bus | 1'b0` y `bus | 1'b1`.
- En el codificador del ejercicio anterior, explicar la diferencia entre usar X para señalar una entrada inválida durante simulación y dejar una salida sin asignar accidentalmente.
- Enumerar dos errores que podrían quedar ocultos si un testbench tratara X como si fuera siempre 0.

10.

Defina $p \text{ mid } q = \neg(p \cdot q)$, es decir, la salida de una compuerta NAND.

- Construir NOT usando una única NAND.
- Construir AND usando únicamente NAND.
- Construir OR usando únicamente NAND.
- Reemplazar cada compuerta del circuito $f(a, b, c) = a \cdot b + \neg a \cdot c$ por las construcciones anteriores, de modo que la implementación final contenga solamente NAND.
- Verificar con una tabla de verdad que ambas versiones de f coinciden.
- Justificar por qué poder construir NOT, AND y OR prueba que NAND permite implementar cualquier función booleana.